

Reinforcement Learning Based PID Parameter Tuning and Estimation for Multirotor UAVs

Serhat Sonmez, Simone Martini, Mathew J. Rutherford, and Kimon P. Valavanis

D. F. Ritchie School of Engineering and Computer Science University of Denver Denver, CO 80208

e-mails: {<firstname>.<lastname>@du.edu}

Abstract—A reinforcement learning (RL) based approach is proposed for PID controller fine-tuning and parameter estimation for effective and accurate tracking of a helix trajectory considering realistic flight controller sampling times. RL exploits a Deep Deterministic Policy Gradient (DDPG) algorithm, which is an off-policy actor-critic method. The quadrotor model follows the Newton-Euler formulation and accounts for complete gyroscopic and drag effects. Training and simulation studies are performed using Matlab/Simulink. Performance evaluation and comparison studies are detailed between the hand-tuned, RL-based tuned, and RL-based full estimation of parameters approaches. Results show that full estimation of controller parameters achieves the smallest attitude and position errors, and that both RL-based strategies significantly improve tracking performance compared to the hand-tuned approach.

Index Terms—Reinforcement learning, PID controller, Multirotor UAVs

I. INTRODUCTION

During the last decade, Unmanned Aerial Vehicles (UAVs) have been widely used for a plethora and wide spectrum of civilian and public domain applications [1], [2], [3], [4], [5], [6]. Multirotor UAVs, mostly quadrotors, have been, and are, widely used due to their VTOL and hovering capabilities.

Several model-based linear, linearized, and nonlinear controllers have been derived and implemented for multirotor navigation and control. However, recently, learning-based algorithms and techniques have gained momentum because they add to the model-based approaches and enhance multirotor modeling and control offering alternatives, among other things, to parameter tuning and estimation, learning, modeling, and understanding the environment.

Representative published surveys on developing and adopting machine learning (ML), deep learning (DL), or reinforcement learning (RL) algorithms for UAV modeling and control include [7], [8], [9], [10], [11], while the recently completed survey in [12] focuses on online learning based multirotor navigation and control.

This research centers around RL-based adaptive tuning and estimation of controller parameters. A hierarchical control structure is adopted to the derived quadrotor Newton-Euler (N-E) mathematical model. The inner loop control parameters are adaptively tuned and estimated using the DDPG algorithm. A comparative study in terms of position and attitude root mean square error (RMSE) is done for the hand-tuned, RL-based tuned and RL-based estimated controller parameters.

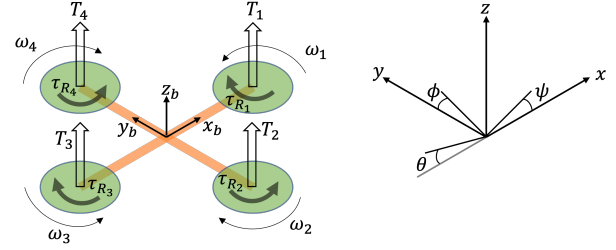


Fig. 1. Inertial and body-fixed frame, quadrotor forces and torques.

The rest of the paper is organized as follows. Section II provides background information and the proposed framework about deriving the complete quadrotor mathematical model, hierarchical PID controller design, and RL. Section III summarizes the combined controller - RL for tuning and estimation purposes. Results of the hand-tuned, RL-based tuned, and RL-based estimated controller performance are compared and evaluated in Section IV. Conclusions are offered in Section V.

II. PROPOSED MODELING AND CONTROL FRAMEWORK

This section briefly summarizes quadrotor modeling and control, and the fundamentals of RL. This information is essential for a better understanding of subsequent developments.

A. Complete Quadrotor Mathematical Model

The classical quadrotor configuration is shown in Fig. 1, in which two rotors rotate clockwise and two counterclockwise. The inertial frame is denoted by $E = \{x, y, z\}$, and the body-fixed frame is denoted by $B = \{x_b, y_b, z_b\}$. Each rotor produces forces and torques that steer the dynamics of the UAV. The linear transformation that maps the propeller rates to the generated thrust T and torques $\tau = [\tau_\phi, \tau_\theta, \tau_\psi]^T$ expressed in B , is modeled as

$$\begin{bmatrix} T \\ \tau_\phi \\ \tau_\theta \\ \tau_\psi \end{bmatrix} = \begin{bmatrix} k & k & k & k \\ 0 & -kl & 0 & kl \\ -kl & 0 & kl & 0 \\ b & -b & b & -b \end{bmatrix} \begin{bmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{bmatrix} \quad (1)$$

where ω_i is the i^{th} propeller rate, k and b are the thrust and rotor reaction drag coefficients, and l represents the projection of the distance between the airframe center of gravity (CG)

and the forces created by rotors on the x_B or y_B axis in the body-fixed frame.

The quadrotor conventional Newton-Euler (N-E) equations of motion are

$$\dot{\xi} = v \quad (2)$$

$$\dot{v} = -g\hat{z} + \frac{1}{m}TR\hat{z} \quad (3)$$

$$\dot{R} = RS(\omega^B) \quad (4)$$

$$I_f\dot{\omega}^B = -\omega^B \times (I_f\omega^B) + \tau \quad (5)$$

where $\xi = (x, y, z)^T$ and $v = (v_x, v_y, v_z)^T$ express the position and linear velocity, respectively, of the body-fixed frame B with respect to the inertial frame E . Gravitational acceleration and mass are defined by g and m , respectively. $\hat{z} = (0, 0, 1)^T$ is the z -axis unit vector and R is the rotation matrix parameterized using the Euler angles defined as

$$R(\eta) = R_z(\psi)R_x(\phi)R_y(\theta) \quad (6)$$

$$R = \begin{bmatrix} c_\theta c_\psi & s_\phi s_\theta c_\psi - c_\phi s_\psi & c_\phi s_\theta c_\psi + s_\phi s_\psi \\ c_\theta s_\psi & s_\phi s_\theta s_\psi + c_\phi c_\psi & c_\phi s_\theta s_\psi - s_\phi c_\psi \\ -s_\theta & s_\phi c_\theta & c_\phi c_\theta \end{bmatrix} \quad (7)$$

where $\eta = (\phi, \theta, \psi)$, $c_\alpha \triangleq \cos \alpha$, $s_\alpha \triangleq \sin \alpha$, $\alpha \in \{\phi, \theta, \psi\}$. R maps vectors in the body-fixed frame to the inertial frame, and $R(\eta) \in SO(3)$. ω^B in (4) represents angular velocities in the body-fixed frame expressed by $\omega^B = (p, q, r)^T$. $S(\cdot)$ in (4) denotes a skew-symmetric matrix, and $S(\omega^B)$ is defined as

$$S(\omega^B) = \begin{bmatrix} 0 & -r & q \\ r & 0 & -p \\ -q & p & 0 \end{bmatrix} \quad (8)$$

Multiplication of $S(\omega^B)$ with a vector η is the cross product, $\omega^B \times \eta$. I_f in (4) is the symmetric positive definite inertia matrix with respect to the body-fixed frame (at the airframe CG).

To increase model accuracy, gyroscopic and drag effects must be considered, and they are taken into account [13]. The gyroscopic effect is the resulting torque induced by the coupling of the quadrotor's angular velocity with the spinning propellers; it is modelled as

$$G_a = \sum_{i=1}^4 (-1)^{i+1} \omega^B \times I_r \vec{\omega}_i \quad (9)$$

where $\vec{\omega}_i = (0, 0, \omega_i)^T$, and $I_r \in \mathbb{R}^{3 \times 3}$ denotes the moment of inertia of the rotor.

A complete drag profile is also included in the model. Total drag is composed of induced drag, translational drag, profile drag, and parasitic drag. The former three act on the propeller blades while the latter is applied on the quadrotor center of gravity (CG). Defining V_p as the translational velocity of the propeller and the drag coefficients K_I, K_T, K_p and K_{par} , the drag effect is modeled as shown below.

Induced Drag: When blade flapping occurs in the rotors, it causes a backward inclination of the force in the rotor. This backward inclination creates the induced drag, represented by

$$D_I = K_I V_p \quad (10)$$

Translational Drag: Airflow passing through the rotor induces a drag, also known as momentum drag [14]. Translational drag is expressed for relatively low velocities as

$$D_T = K_T V_p \quad (11)$$

Profile Drag: When a quadrotor moves horizontally through the air, the transverse airflow through the rotor blades creates the profile drag. The angle of attack of the blades' airfoil does not change the profile drag, but the airspeed changes it in a limited way. Profile drag is modeled as

$$D_p = K_p V_p \quad (12)$$

Parasitic Drag: Parasitic drag is created from the air friction to the airframe and rotors, and it is given by

$$D_{par} = K_{par} |v| v \quad (13)$$

Drag effects result in an additional drag force, D_b , and torque, τ_{drag} , as shown in [13].

Therefore, the revised, complete, quadrotor N-E equations of motion that include gyroscopic and drag effects are rewritten as

$$\dot{\xi} = v \quad (14)$$

$$\dot{v} = -g\hat{z} + \frac{1}{m}TR\hat{z} - \frac{1}{m}RD_b \quad (15)$$

$$\dot{R} = RS(\omega^B) \quad (16)$$

$$I_f\dot{\omega}^B = -\omega^B \times (I_f\omega^B) - G_a + \tau + \tau_{Drag} \quad (17)$$

Next, the PID controller configuration is detailed.

B. PID Based Hierarchical Control Configuration

PID controllers have been widely adopted and used for multirotor navigation and control because they reduce errors and accumulated past errors (preventing steady-state errors), improve system stability, anticipate future errors by considering the rate of change, and enhance ability to respond quickly to disturbances [15].

For underactuated systems, like quadrotor UAVs, a hierarchical control configuration is usually employed, composed of an outer position control loop and an inner attitude control loop. The outer loop produces thrust and desired Euler angles, while the inner loop provides desired torques.

In the authors' previous work [16], inner and outer loop control actions were derived considering the Euler-Lagrange (E-L) formulation. In this paper, control actions are produced following the Newton-Euler (N-E) formulation.

Outer Loop: The outer loop PID control law is formulated as

$$\dot{v} = \dot{v}_d + K_{P\xi}(\xi_d - \xi) + K_{I\xi} \int (\xi_d - \xi) dt + K_{D\xi}(\dot{v}_d - \dot{v}). \quad (18)$$

with K_{P_ξ} , K_{I_ξ} and K_{D_ξ} being 3×3 control gain diagonal matrices to be tuned. Using the same approach as in [16] and [17], consider the virtual control signal $U = \dot{v} = (U_1, U_2, U_3)^T$ and define a second virtual control signal compensating for the drag force component

$$U^* = U + \frac{1}{m} R D_b \quad (19)$$

Following the derivation in [18], the desired pitch, desired roll, and control thrust are given by, respectively

$$\theta_d = \arctan \left(\frac{U_1^* \cos(\psi_d) + U_2^* \sin(\psi_d)}{U_3^* + g} \right) \quad (20)$$

$$\phi_d = \arcsin \left(\frac{U_1^* \sin(\psi_d) - U_2^* \cos(\psi_d)}{\sqrt{U_1^{*2} + U_2^{*2} + (U_3^* + g)^2}} \right) \quad (21)$$

$$T_d = m((s_\theta c_\phi c_\psi + s_\phi s_\psi)U_1^* + (s_\theta s_\psi c_\theta - s_\phi c_\psi)U_2^* + c_\phi c_\theta(U_3^* + g)) \quad (22)$$

Inner Loop: The inner loop control law is designed as a PID with feedback linearization. The dynamic compensation includes the aerodynamic drag torques and is given as

$$\tau_d = I_f y + \omega^B \times I_f \omega^B + G_a - \tau_D \quad (23)$$

where y is the virtual control vector defined as

$$y = \dot{\omega}_d^B + K_{P_\eta}(\eta_d - \eta) + K_{I_\eta} \int (\eta_d - \eta) dt + K_{D_\eta}(\dot{\eta}_d - \dot{\eta}) \quad (24)$$

where K_{P_η} , K_{I_η} and K_{D_η} being 3×3 control gain diagonal matrices to be tuned. Consequently, the proposed control scheme includes a cascaded PID controller with feedback linearization.

C. Reinforcement Learning

RL centers on training an agent that decides about taking actions by maximizing a long-term benefit through trial and error. RL is generally described by a Markov Decision Process (MDP). The agent-environment interaction in an MDP is illustrated in Fig. 2. Agent, environment, and action represent the controller, controlled system, and control signal in engineering terms, respectively [19].

A DDPG algorithm is adopted that is designed to handle high-dimensional action spaces. It is an off-policy actor-critic

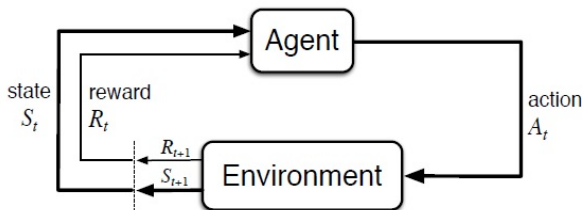


Fig. 2. Interaction scheme between the agent and environment [19].

algorithm that works based on the expected gradient of the action-value function that is given by

$$q_\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s, A_t = a \right] \quad (25)$$

where $q_\pi(s, a)$ denotes the *action-value function* for policy π at state s and action a . $\mathbb{E}_\pi[\cdot]$ represents the expected value under policy π . $\sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$ is the sum of discounted future rewards starting from time t in state s and represents the expected *discounted return*, and γ is the *discount rate*, $0 \leq \gamma \leq 1$ [12] [19] [20].

The DDPG algorithm finds a deterministic target policy using an exploratory behavior policy. Thus, it outputs a specific action rather than a probability distribution over actions.

The actor-critic approach includes both a value function-based and a policy search-based method. While the actor refers to the policy search-based method and chooses actions in the environment, the critic refers to the value function-based method and evaluates the actor using the value function.

Two different state spaces S , action space A , and reward functions r , are used because of controller parameter tuning and estimation. The dimensions of the state space change, while the dimensions of the action space remain the same. This is further explained in the next section along with the reward function differences. Moreover, the next section details the proposed method to tune and estimate controller parameters.

III. COMBINING CONTROLLER AND REINFORCEMENT LEARNING

It is essential to properly adjust controller parameters as this directly impacts performance. Given a PID controller, an excessively high proportional gain may lead to overshooting, an insufficient integral gain may result in a steady-state error, and a very high derivative gain may lead to instabilities. Thus, achieving an 'optimal balance' among the three gains is crucial to ensure a PID controller's accurate and stable response (when applied to different dynamic systems).

The MATLAB/Simulink environment has been used for the presented simulation studies. While Simulink offers a good tool for PID controller tuning, challenges arise due to difficulties in linearizing the plant (using Simulink). Consequently, manual tuning of PID parameters is performed.

A. PID Controller Parameter Tuning

The starting step is manual tuning of controller parameters. This is followed by employing RL to further refine manual tuning. The block diagram configuration of controller parameters tuning using RL is illustrated in Fig. 3. The overall system consists of three components: controller, plant (quadrotor), and agent (RL component). The overall configuration of Fig. 3 includes a trajectory planner, the linear transformation (1) and parameter tuning parts. The notation has already been explained.

The state space S (for tuning) includes positions (ξ) and velocities ($\dot{\xi}$) along the x, y, and z axes, as well as Euler angles (η) and their corresponding rates ($\dot{\eta}$). It also includes

position and velocity errors ($e_\xi, e_{\dot{\xi}}$) along the x, y, and z axes, errors in Euler angles and their rates ($e_\eta, e_{\dot{\eta}}$). The agent learns to dynamically adjust the normalized weights ($n_{P_\eta}, n_{I_\eta}, n_{D_\eta}$), which create the action space ($A \in \mathbb{R}^9$) within the range of $[-1, 1]$ for all controller parameters. The state and action spaces are denoted as follows

$$\begin{aligned} S &= [\xi, \dot{\xi}, \eta, \dot{\eta}, e_\xi, e_{\dot{\xi}}, e_\eta, e_{\dot{\eta}}] \in \mathbb{R}^{24} \\ A &= [n_{P_\eta}, n_{I_\eta}, n_{D_\eta}] \in \mathbb{R}^9 \end{aligned} \quad (26)$$

Considering the normalized weights, the corresponding parameter tuning equations are

$$\begin{aligned} K_{P_{\eta, new}} &= K_{P_\eta} + a n_{P_\eta} K_{P_\eta} \\ K_{I_{\eta, new}} &= K_{I_\eta} + a n_{I_\eta} K_{I_\eta} \\ K_{D_{\eta, new}} &= K_{D_\eta} + a n_{D_\eta} K_{D_\eta} \end{aligned} \quad (27)$$

where $K_{P_{\eta, new}}, K_{I_{\eta, new}}, K_{D_{\eta, new}}$ are the RL fine-tuned proportional, integral, and derivative gains (parameters), respectively. a is the search rate with an assigned value of 0.5. The manually tuned inner loop controller parameters serve as the "base" gains. The combination of the search rate and the normalized weights helps tune controller parameter values within the interval of $[-50\%, 50\%]$ relative to the base controller gains.

Since RL is then applied to further tune the inner loop controller gains, the reward function is defined as a function of the attitude error norm. Considering simulation studies after manually tuned gains, it is observed that the norm of the attitude error ranges from 10^{-4} to 10^{-2} rads. Given this relatively wide range, the reward function is set as a piece-wise function of the attitude error norm as shown in (28)

$$R(e_\eta) = \begin{cases} -25, & 0.04 < \|e_\eta\| \\ -15, & 0.02 < \|e_\eta\| \leq 0.04 \\ -10, & 0.01 < \|e_\eta\| \leq 0.02 \\ -5, & 0.005 < \|e_\eta\| \leq 0.01 \\ -1, & 0.001 < \|e_\eta\| \leq 0.005 \\ 10, & \|e_\eta\| \leq 0.001 \end{cases} \quad (28)$$

The choice of the highest critical value as 0.04 rad reflects the observation that the norm of the attitude error surpasses this threshold within (the first) 5s.

For training, the complete N-E model shown in (14)-(17) is used. Given the considerable duration of the training process, the simulation period is set to be 20s. This does not affect training as it has been observed that the dynamics stabilize much sooner. The criterion to complete training is based on achieving an average reward. The target average reward value is determined by analyzing the distribution of step counts across the error intervals shown in (28) relative to the norm of the attitude error observed with the manually tuned parameters. The step count distribution returns the values of [16, 31, 13, 5, 67, 269]. Using this distribution, the target average reward value is computed as 1603. To enhance performance, the average reward value is then adjusted to 1843, which represents a 15% increase over the baseline average reward of 1603.

TABLE I
HYPERPARAMETERS

Parameters	Tuning	Estimation
Sampling time	0.05	0.05
Reward discount factor γ	0.99	0.99
Learning rate for actor	10^{-3}	10^{-3}
Learning rate for critic	10^{-3}	10^{-3}
L_2 Regularization factor	10^{-5}	10^{-4}
Optimizer parameter ϵ	10^{-8}	10^{-8}
Minimum batch size	1024	64
Experience buffer length	10^6	10^6

The underlying actor and critic neural network (NN) structures are depicted in Fig. 4 and 5, respectively. The *tanh* activation function is used in the output of the actor, and the agent output is set between $[-1, 1]$. The training phase parameters to tune controller gains based on RL are shown in Table I.

The next section details PID parameter estimation based on RL.

B. PID Parameter Estimation

The aim is to estimate the inner loop attitude controller parameters. A DDPG algorithm is, again, used for the estimation process. The block diagram configuration of the RL-based parameter estimation is shown in Fig. 6.

The agent is trained to estimate the PID controller parameters. In this case, for this task, the state space (S) includes attitude angles and corresponding errors, while the action space (A) consists of the diagonal elements of the inner loop proportional, integral, and derivative gain matrices ($K_{P_\eta}, K_{I_\eta}, K_{D_\eta}$). The state space is represented as six-dimensional, and the action space as nine-dimensional, as follows

$$\begin{aligned} S &= [\eta, e_\eta] \in \mathbb{R}^6 \\ A &= [K_{P_k}, K_{I_k}, K_{D_k}] \in \mathbb{R}^9 \end{aligned} \quad (29)$$

where k denotes ϕ, θ , and ψ .

The reward function for parameter estimation has the following form

$$R = -\|e_\eta\|^2 - 0.1\|\tau_{d_{prev}}\|^2 - 250D \quad (30)$$

where $\tau_{d_{prev}}$ denotes the previous control action of the inner loop controller, and D denotes the Done function, defined as

$$D = \begin{cases} 1, & |e_{\xi_i}| \geq 1 \vee |e_{\eta_k}| \geq 1 \wedge t < T_f \\ 0, & \text{otherwise} \end{cases} \quad (31)$$

where, t is the simulation time, T_f is the final time, i denotes x, y or z, and k represents ϕ, θ or ψ . This implies that the D function is activated once, if and only if $|e_{\xi_i}|$ or $|e_{\eta_k}|$ surpass the given thresholds before the episode is completed.

In RL, the exploration and exploitation dilemma is a fundamental problem. An agent improves action selection over time by balancing the exploration of new options with the exploitation of known strategies based on past experience [19], [21]. During the training process, the Done function acts as a termination condition for each episode. Without using D in the reward function, the agent receives a higher reward when

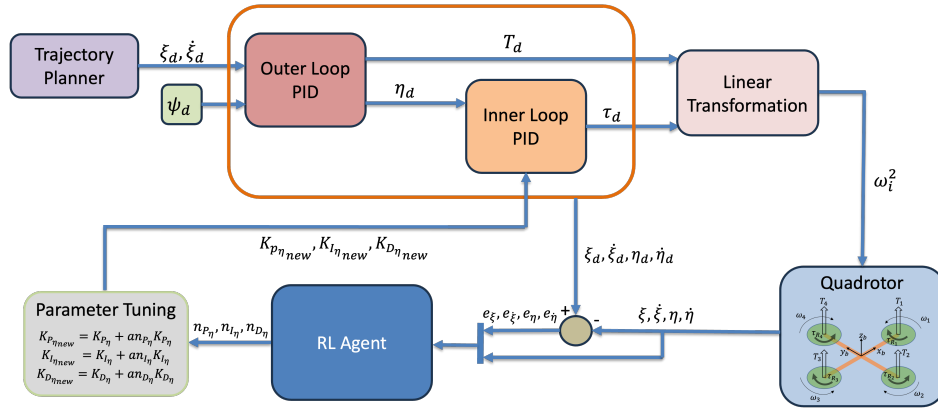


Fig. 3. Block diagram of the PID parameter tuning based on RL.



Fig. 4. Structure of the actor neural network used for PID parameter tuning.

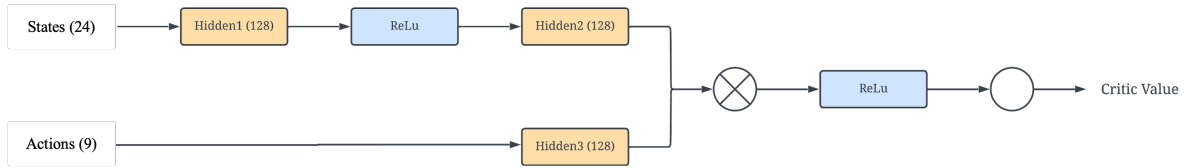


Fig. 5. Structure of the critic neural network used for PID parameter tuning.

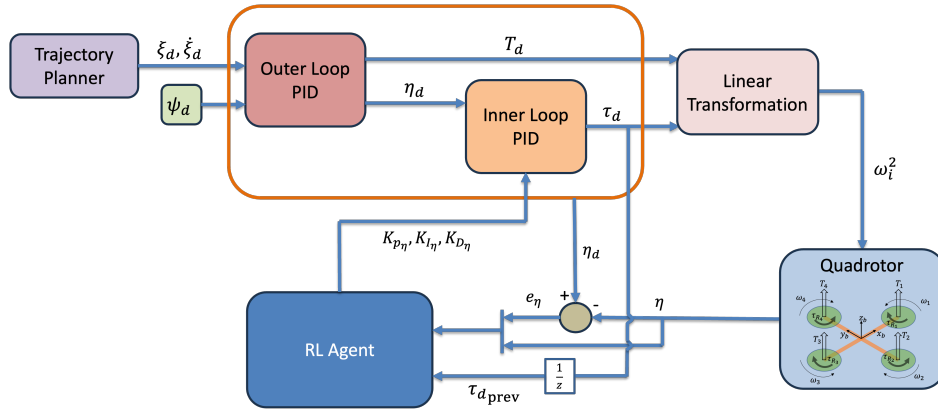


Fig. 6. Block diagram of PID parameter estimation based on RL.



Fig. 7. Structure of the actor neural network used in PID parameter Estimation.

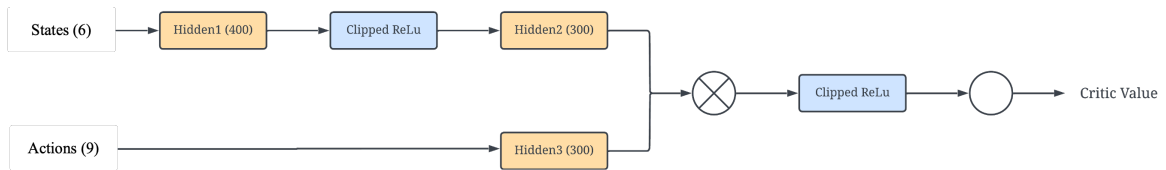


Fig. 8. Structure of the critic neural network in PID parameter estimation.

TABLE II
UAV AND CONTROLLER PARAMETERS

Param.	Value	Name
m	1.04 kg	Total mass
l	0.2 m	Quadrotor arm length
I_x	$7.691 \times 10^{-3} \text{ kgm}^2$	Total moment of inertia along x_B
I_y	$7.691 \times 10^{-3} \text{ kgm}^2$	Total moment of inertia along y_B
I_z	$15.254 \times 10^{-2} \text{ kgm}^2$	Total moment of inertia along z_B
I_{r_x}	$1.406 \times 10^{-5} \text{ kgm}^2$	Rotor moment of inertia along x_{rot}
I_{r_y}	$1.406 \times 10^{-5} \text{ kgm}^2$	Rotor moment of inertia along y_{rot}
I_{r_z}	$2.813 \times 10^{-5} \text{ kgm}^2$	Rotor moment of inertia along z_{rot}
k	1.149×10^{-5}	Thrust coefficient
b	2.690×10^{-8}	Rotor reaction drag coefficient
K_I	$5 \times 10^{-3} \text{ kgs}^{-1}$	Induced drag coefficient
K_T	$1 \times 10^{-3} \text{ kgs}^{-1}$	Translational drag coefficient
K_p	$2 \times 10^{-3} \text{ kgs}^{-1}$	Profile drag coefficient
K_{par}	$7 \times 10^{-4} \text{ kgs}^{-1}$	Parasitic drag coefficient
K_{P_ξ}	$diag(6, 6, 10)$	Position proportional controller gain
K_{I_ξ}	$diag(4, 4, 8)$	Position integral controller gain
K_{D_ξ}	$diag(4, 4, 6)$	Position derivative controller gain
K_{P_η}	$diag(5, 5, 5)$	Attitude proportional controller gain
K_{I_η}	$diag(2, 2, 0.5)$	Attitude integral controller gain
K_{D_η}	$diag(2, 2, 2)$	Attitude derivative controller gain

it fails at the beginning of an episode compared to failing after 2s. This causes the aforementioned dilemma. To resolve the dilemma, the used Done function prevents the agent from repeating the same failing output. In this manner, the reward function is designed for punishment and behaves as a cost function. Thus, the long-term aim of the agent is for the reward to converge to zero.

Training phase parameters for controller gain estimation are shown in Table I. The actor and critic NN structures are displayed in Fig. 7 and 8, respectively. A clipped ReLu activation function is used in the output of the actor NN, thus, the agent estimates the control parameters in the set range of [0, 20000].

IV. SIMULATION RESULTS

For training and simulation studies a Matlab/Simulink environment is used, implementing the Deep Learning and Reinforcement Learning Toolbox. Training and simulation run over 20s and 100s, respectively, tracking the helix trajectory of Fig. 9. The chosen quadrotor parameters are adopted from published literature [22], [23]; they are reported in Table II along the manually tuned controller parameters.

The trained agent, for the tuning task, is applied to the simulation and tunes the controller parameters online. Hence, the controller parameters change in time and they are displayed in Fig. 10. The agent changes the parameters in the first 1.6s, and then all parameters become constant. For the estimation task, Fig. 11 shows the trained agent estimated control parameters in time.

Next, the performance of the manually tuned, RL-based-tuned, and RL-based-estimated controllers are compared. The attitude and position errors and error norms are shown in Fig. 12-15, respectively. Note that, since the outer loop produces the desired Euler angle references, the initial error of the attitude dynamics is not zero.

The root mean square errors (RMSE) of the normalized attitude and position trajectory are calculated and the values are provided in Table III. Findings indicate that, compared to the manually tuned controller, the RL-based tuned controller achieves 20.45% smaller RMSE, while the RL-based estimation method results in a 39.13% performance gain in terms of the RMSE value of the attitude. Additionally, the RL-based tuned controller exhibits a 27.66% improvement over the manually tuned controller, while the RL-based estimation

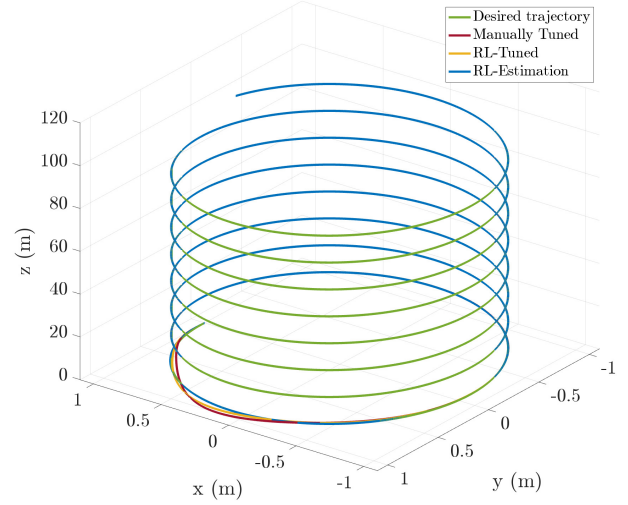


Fig. 9. Desired helix x, y, z -trajectory and the trajectory of the quadrotor with the manually tuned, RL-based tuned, and RL-based estimated PID

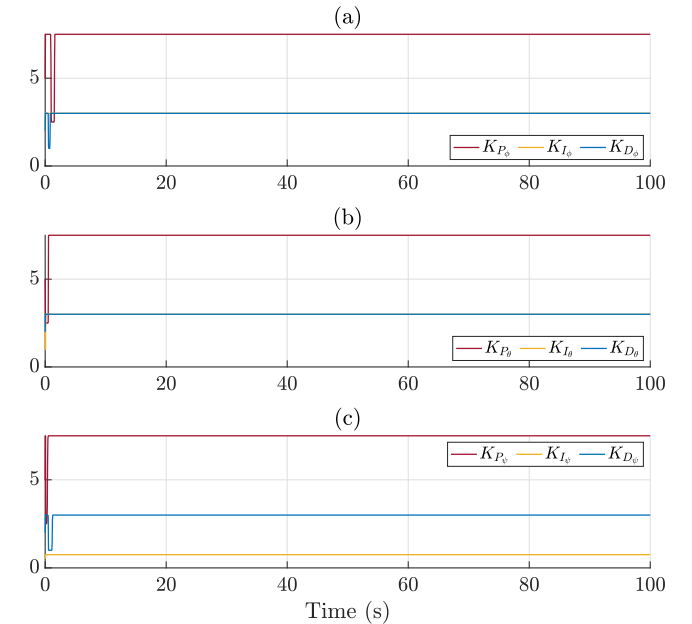


Fig. 10. The tuned parameters based on RL in time: (a) The parameters for roll angle ϕ , (b) The parameters for pitch angle θ , (c) The parameters for the yaw angle ψ .

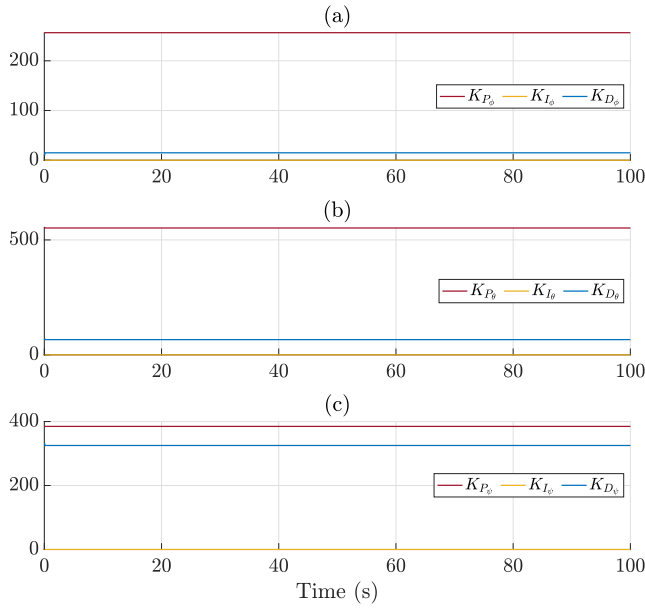


Fig. 11. The estimated parameters based on RL in time: (a) The parameters for roll angle ϕ , (b) The parameters for pitch angle θ , (c) The parameters for the yaw angle ψ .

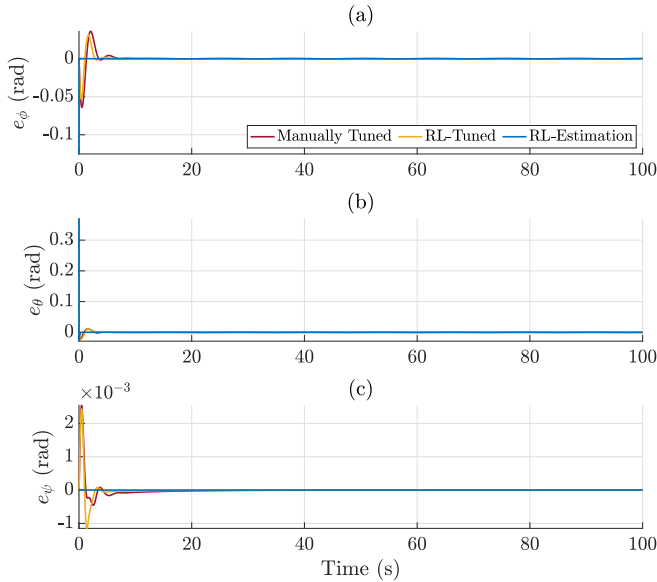


Fig. 12. Attitude errors: (a) roll angle ϕ , (b) pitch angle θ , (c) yaw angle ψ .

method performance shows a substantial 63.99% increase in the RMSE of the normalized position.

Finally, the estimated controller gains converge to a constant value, and the integral gain is set to zero by the agent. Thus, the controller effectively behaves as a PD controller. This suggests that by compensating for external disturbances (i.e., gyroscopic and aerodynamic effects) in the feedback linearization, a PD solution is able to achieve zero steady-state error. Moreover, by eliminating the integral control action, the resulting attitude dynamics show a less oscillatory behavior, as verified in Fig. 12 and 14.

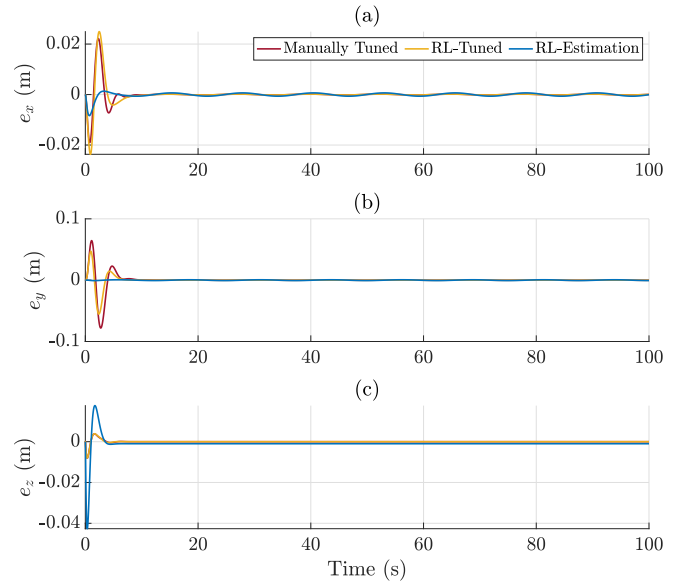


Fig. 13. Position errors: (a) x -axis, (b) y -axis, (c) z -axis.

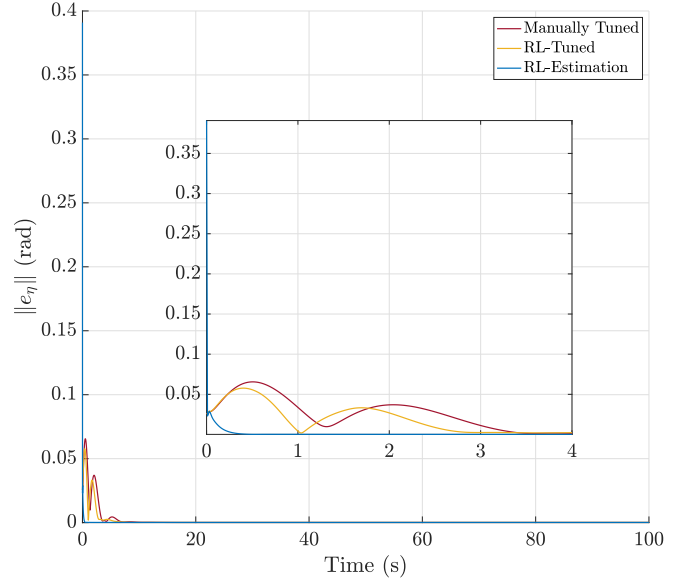


Fig. 14. The attitude error norms of the manually tuned, RL-based tuned, and RL-based estimated controllers.

TABLE III
RMSES OF ATTITUDE AND POSITION

	$RMSE_\eta$	$RMSE_\xi$
Hand-Tuned	$6.6 \times 10^{-3} \text{ rad}$	$10.3 \times 10^{-3} \text{ m}$
Tuning	$5.2 \times 10^{-3} \text{ rad}$	$7.4 \times 10^{-3} \text{ m}$
Estimation	$4 \times 10^{-3} \text{ rad}$	$3.7 \times 10^{-3} \text{ m}$

V. CONCLUSION

In this paper, a DDPG-based RL algorithm has been utilized for PID controller fine-tuning and parameter estimation; the method is implemented and tested to track a helix trajectory, which reflects a realistic flight scenario. The key points and contributions of this study is the derivation of a comprehensive

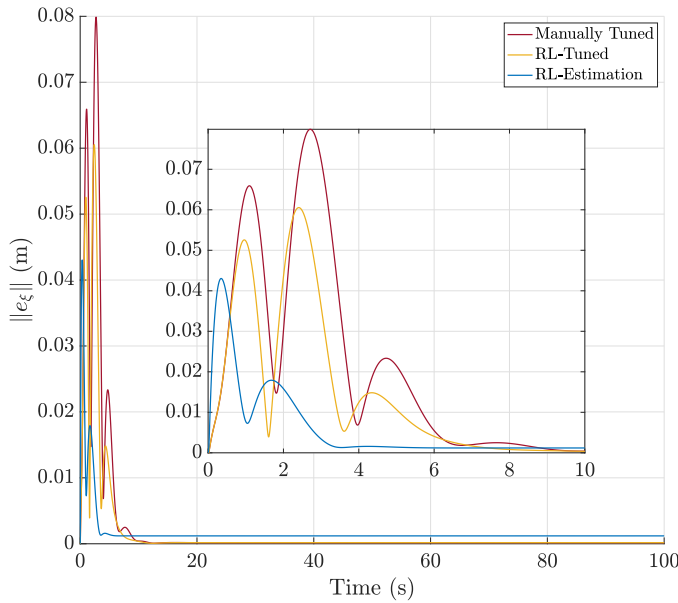


Fig. 15. The position error norms of the manually tuned, RL-based tuned, and RL-based estimated controllers.

quadrotor model that includes gyroscopic, drag and aerodynamic effects, which are considered in the training phase of the agent. By leveraging the DDPG algorithm, an off-policy and actor-critic method, a robust and effective means of optimizing controller performance has been achieved. Performance evaluation and comparison of the manually tuned, RL-based tuned, and RL-based estimated PID controllers reveals that the RL-based estimated controller achieves the best performance. Although the targeted average reward value is increased by 15% for the RL-based PID parameter tuning as mentioned in Section III-A, the performance of the RL-based tuned controller reaches 20.45% and 27.66% in terms of the RMSE values of the attitude and position, respectively. On the other hand, the RL-based estimator has converted the controller, designed as PID, to PD setting the integral component of the parameters as zero. In conclusion, the proposed DDPG-based RL approach presents a viable method to tune and estimate PID controller parameters with substantial performance improvements.

ACKNOWLEDGMENT

This research has been partially supported by the Ministry of National Education of the Republic of Turkey on behalf of the Istanbul Medeniyet University, Turkey, and the D. F. Ritchie School of Engineering and Computer Science, University of Denver, CO 80208.

REFERENCES

- [1] C. Martinez, C. Sampedro, A. Chauhan, and P. Campoy, "Towards autonomous detection and tracking of electric towers for aerial power line inspection," in *2014 international conference on unmanned aircraft systems (ICUAS)*, pp. 284–295, IEEE, 2014.
- [2] H. Ren, Y. Zhao, W. Xiao, and Z. Hu, "A review of uav monitoring in mining areas: Current status and future perspectives," *International Journal of Coal Science & Technology*, vol. 6, pp. 320–333, 2019.

- [3] M. A. Olivares-Mendez, C. Fu, P. Ludvig, T. F. Bissyandé, S. Kannan, M. Zurad, A. Annaiyan, H. Voos, and P. Campoy, "Towards an autonomous vision-based unmanned aerial system against wildlife poachers," *Sensors*, vol. 15, no. 12, pp. 31362–31391, 2015.
- [4] R. Bassoli, C. Sacchi, F. Granelli, and I. Ashkenazi, "A virtualized border control system based on uavs: Design and energy efficiency considerations," in *2019 IEEE aerospace conference*, pp. 1–11, IEEE, 2019.
- [5] A. Carrio, J. Pestana, J.-L. Sanchez-Lopez, R. Suarez-Fernandez, P. Campoy, R. Tendero, M. García-De-Viedma, B. González-Rodrigo, J. Bonatti, J. G. Rejas-Ayuga, *et al.*, "Ubristes: Uav-based building rehabilitation with visible and thermal infrared remote sensing," in *Robot 2015: Second Iberian Robotics Conference: Advances in Robotics, Volume 1*, pp. 245–256, Springer, 2016.
- [6] L. Li, Y. Fan, X. Huang, and L. Tian, "Real-time uav weed scout for selective weed control by adaptive robust control and machine learning algorithm," in *2016 ASABE Annual International Meeting*, p. 1, American Society of Agricultural and Biological Engineers, 2016.
- [7] A. Carrio, C. Sampedro, A. Rodriguez-Ramos, and P. Campoy, "A review of deep learning methods and applications for unmanned aerial vehicles," *Journal of Sensors*, vol. 2017, 2017.
- [8] A. S. Polydoros and L. Nalpantidis, "Survey of model-based reinforcement learning: Applications on robotics," *Journal of Intelligent & Robotic Systems*, vol. 86, no. 2, pp. 153–173, 2017.
- [9] S. Y. Choi and D. Cha, "Unmanned aerial vehicles using machine learning for autonomous flight: state-of-the-art," *Advanced Robotics*, vol. 33, no. 6, pp. 265–277, 2019.
- [10] A. T. Azar, A. Koubaa, N. Ali Mohamed, H. A. Ibrahim, Z. F. Ibrahim, M. Kazim, A. Ammar, B. Benjdira, A. M. Khamis, I. A. Hameed, *et al.*, "Drone deep reinforcement learning: A review," *Electronics*, vol. 10, no. 9, p. 999, 2021.
- [11] L. Brunke, M. Greeff, A. W. Hall, Z. Yuan, S. Zhou, J. Panerati, and A. P. Schoellig, "Safe learning in robotics: From learning-based control to safe reinforcement learning," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 5, pp. 411–444, 2022.
- [12] S. Sönmez, M. J. Rutherford, and K. P. Valavanis, "A survey of offline and online learning-based algorithms for multirotor uavs," 2024.
- [13] M. Bangura, R. Mahony, *et al.*, "Nonlinear dynamic modeling for high performance control of a quadrotor," 2012.
- [14] J.-M. Pfimlin, P. Binetti, P. Soueres, T. Hamel, and D. Trouchet, "Modeling and attitude control analysis of a ducted-fan micro aerial vehicle," *Control Engineering Practice*, vol. 18, no. 3, pp. 209–218, 2010.
- [15] C. Knospe, "Pid control," *IEEE Control Systems Magazine*, vol. 26, no. 1, pp. 30–31, 2006.
- [16] S. Martini, S. Sönmez, A. Rizzo, M. Stefanovic, M. J. Rutherford, and K. P. Valavanis, "Euler-lagrange modeling and control of quadrotor uav with aerodynamic compensation," in *2022 International Conference on Unmanned Aircraft Systems (ICUAS)*, pp. 369–377, IEEE, 2022.
- [17] S. Martini, M. Stefanovic, A. Rizzo, M. J. Rutherford, P. Liveri, and K. P. Valavanis, "A benchmark framework for testing, evaluation, and comparison of quadrotor linear and nonlinear controllers," in *2023 International Conference on Unmanned Aircraft Systems (ICUAS)*, pp. 471–478, IEEE, 2023.
- [18] Z. Zuo, "Trajectory tracking control design with command-filtered compensation for a quadrotor," *IET control theory & applications*, vol. 4, no. 11, pp. 2343–2355, 2010.
- [19] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.
- [20] E. Bilgin, *Mastering reinforcement learning with python: build next-generation, self-learning models using reinforcement learning techniques and best practices*. Packt Publishing Ltd, 2020.
- [21] M. Lapan, *Deep Reinforcement Learning Hands-On: Apply modern RL methods to practical problems of learning, robotics, discrete optimization, web automation, and more*. Packt Publishing Ltd, 2020.
- [22] M. Walid, N. Slaheddine, A. Mohamed, and B. Lamjed, "Modelling, identification and control of a quadrotor uav," in *2018 15th International Multi-Conference on Systems, Signals & Devices (SSD)*, pp. 1017–1022, IEEE, 2018.
- [23] S. Islam, P. Liu, and A. El Saddik, "Nonlinear adaptive control for quadrotor flying vehicle," *Nonlinear Dynamics*, vol. 78, pp. 117–133, 2014.